

第七章 AutoCAD的二次开发

- 7.1 自定义菜单
- 7.2 DXF文件
- 7.3 生成DXF文件

7.1 自定义菜单

1、菜单文件

专业绘图：专业绘图操作，添加自己的菜单可以提高绘图效率。

LISP函数法：加载一个函数，输入一个命令。
——必须记住函数名和命令名称。

菜单法：一次加载，多次使用。
——不用记住函数名和命令名称。
——标注表面粗糙度。

7.1 自定义菜单

1、菜单文件

- **AutocAD**最大的特点是采用了开放大体系结构，允许别户或第三方广家开发自己的命令、标准库文件或应用程序等，以增强软件的功能，提高用户绘图的效率。
- 用户定制**AutoCAD**时，用户化菜单是使用最为广泛的一种定制方式。
- 从菜单的结构上看，一般菜单文件由屏幕菜单，下拉式菜单、图标菜单、定点设备按钮菜单、数字化仪菜单、图像控件菜单和辅助菜单等组成。除了屏幕菜单只能占用一个屏幕区之外，其他的菜单可以占用多个屏幕区。
- 菜单由菜单文件决定，自定义菜单必须了解菜单文件。

7.1 自定义菜单

1、菜单文件

- 由于从**ACAD2006**版起开始使用一种新的菜单文件格式“**.CUI**”，它替代了原来的“**.MNU**”格式的菜单，所以在菜单的自定义的具体操作方面会略有不同，但在本质上是没有任何区别的，而且**ACAD2006**（7）等版本都向前兼容原来的菜单格式，也就是说，你可以按原来的方式编写自定义的菜单文件“***.MNU**”，在**ACAD2006**或更新的版本中以“**MENULOAD**”命令来加载。

7.1 自定义菜单

1、菜单文件

1) 文件类型

- AutoCAD系统的基础菜单文件放在AutoCAD安装目录下的\SUPPORT子目录中，共有五个文件。
- 无论是AutoCAD系统的基础菜单，还是附加（局部）菜单或是用户自己定制的菜单，都是由.mnu/.mnc/.mns/.mnr/.mnl这五个文件组成。这五个文件组成了一个菜单的整体，但也并不是每一个菜单都包括全部的这五个文件。
- 五个文件在菜单中起着各自的作用，但其作用是有主次之分的，其生成过程有主从之分。要使用好菜单，必须明白这五个文件相互之间的关系和在菜单的调用过程中各自所起的作用。

7.1 自定义菜单

1、菜单文件

1) 文件类型

- **.mnu**是模板菜单文件，除了**.mnl**文件外，其余三个文件最初都源出自于它。**AutoCAD**调用时自动对它进行编译，产生对应的**.mnc**和**.mnr**文件以及**.mns**文件。
- **.mnc**是**.mnu**文件编译后的版本，实际上**.mnc**文件是**AutoCAD**真正直接执行的菜单文件。
- **.mnr**也是在**.mnu**文件编译时产生，它存放着菜单所使用的位图之类的菜单资源。
- 最初的**.mns**文件也是**.mnu**文件被编译时由**AutoCAD**自动产生的，它的基本内容和**.mnu**文件类似，只是少了有关的说明和注释行。

7.1 自定义菜单

1、菜单文件

1) 文件类型

- 在.mnu文件被编译时，.mns文件和.mnc文件的菜单内容是一致的，但如果在AutoCAD的操作中，用户对菜单作了更改，这些改动都记录在.mns文件中，所以.mns文件对应的是AutoCAD系统中当前菜单的内容。
- .mnl文件在内容上相对独立一些，它包含由菜单所使用的AutoLISP表达式，如各种命令简化指令的定义，还有层排序、指定层这样的LISP程序。当加载一个菜单文件时，如果系统中存在与菜单文件同名的.mnl文件，则这个.mnl文件也会被加载。

7.1 自定义菜单

1、菜单文件

1) 文件类型

文件类型	说明
.MNU	模板菜单文件
.MNC	编译后的菜单文件。此二进制文件包含定义菜单功能和外观的命令字符串和菜单语法
.MNR	菜单资源文件。此二进制文件包含由菜单使用的位图
.MNS	源菜单文件（由AutoCAD生成）
.MNL	与菜单配套的AutoLISP程序。这些文件包含由菜单文件使用的AutoLISP表达式。当加载同名的菜单文件时，这些文件也载入内存

7.1 自定义菜单

1、菜单文件

1) 文件类型

2) 文件结构

一个文件分几段，段数可多可少。

在菜单中出现的主菜单段都是由三个星号打头。“***段名”标记一段的开始。

段名是专用的，每个名称对应于一种菜单。

主菜单段可包含任意数目的子菜单，子菜单由两个星号打头。

段标识	菜单类型
***MENUGROUP	菜单文件的组名
***BUTTONSn	定点设备的按组菜单
***AUXn	系统定点设备菜单
***POPn	下拉/光标菜单域
***TOOLBARS	工具条菜单定义域
***IMAGE	图象菜单域
***SCREEN	幕菜单域
***TABLETn	图形输入板（数字化仪）菜单域
***HELPSTRINGS	随文帮助文本域
***ACCELERATORS	加速键定义

7.1 自定义菜单

每个POP段有一个菜单名：
“**菜单名”可自己定义。

1、菜单文件

1) 文件类型

2) 文件结构

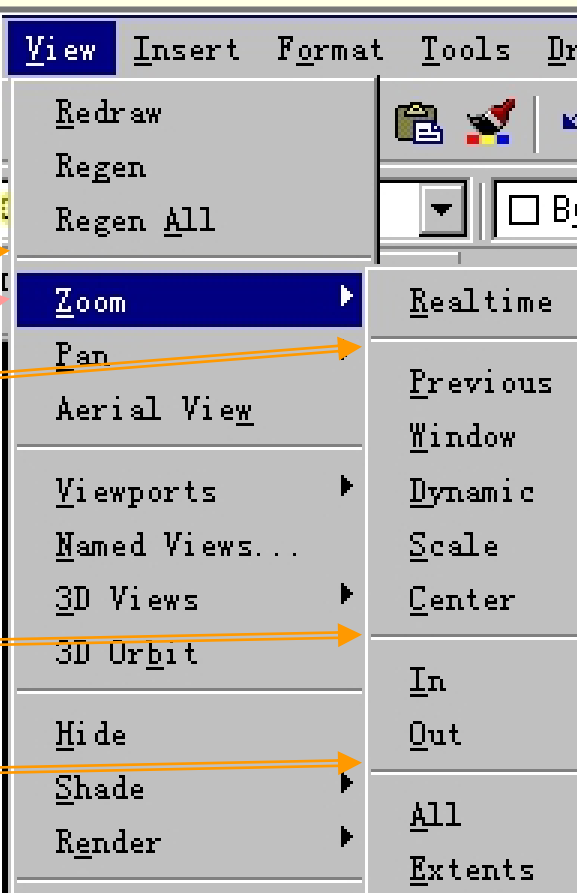
3) POP段结构

菜单外观和级
联菜单外观定义

分割线

每行一个
菜单项

```
***POP3
**VIEW
ID MnView      [&View]
ID Redrawall   [&Redraw]' redrawall
ID Regen       [Re&gen]^C^C_regen
ID Regenall    [Regen &All]^C^C_regenall
ID MnZoom      [-]
ID ZoomRealt   [->&Zoom]
ID ZoomRealt   [-]
ID ZoomPrevi   [-]
ID ZoomWindo   [-]
ID ZoomDynam   [-]
ID ZoomScale   [-]
ID ZoomCente   [-]
ID ZoomIn      [-]
ID ZoomOut     [-]
ID ZoomAll     [-]
ID ZoomExten   [-]
.....
```



7.1 自定义菜单

1、菜单文件

- 1) 文件类型
- 2) 文件结构
- 3) POP段结构
- 4) 菜单项语法

- 菜单项是组成菜单文件的基本要素。菜单项的基本语法在所有支持菜单项的菜单部分中都是一致的。
- 每个菜单项由名称标记、标签和菜单宏组成（除了Image部分不允许名称标记）。
- 一个菜单项通常占文件的一行，格式如下：
菜单名称标记 标签 菜单宏或命令

7.1 自定义菜单

1、菜单文件

- 1) 文件类型
- 2) 文件结构
- 3) POP段结构
- 4) 菜单项语法

```
***POP7
**DRAW
ID_MnDraw      [Draw(&D)]
ID_Line         [Line(&L)] ^C^C_line
```

菜单宏：响应菜单的命令，执行响应的程序，是菜单项的关键组成部分。

标识名：字符串唯一标记、便于连接快捷键、工具条、帮助等

标签：菜单标记显示在屏幕上，用[]进行标记。如果用快捷键，只需在快捷键字母前加上&。

7.1 自定义菜单

在菜单宏中，经常要用到一些特殊的字符：

表 3-3 特殊字符

字符	说 明	字符	说 明
***	菜单标题	^B	切换捕捉开或关 (Ctrl+B)
**	菜单子标题	^C	取消命令 (Esc)
,	产生 Enter	^D	切换坐标显示开或关 (Ctrl+D)
[]	菜单标记	^E	设置下一个等轴测平面 (Ctrl+E)
^M	产生 Enter	^G	切换栅格开或关 (Ctrl+G)
^I	产生 Tab	^H	产生退格
SPACEBAR	输入空格：菜单项中命令序列之间的空格等价于按 SPACEBAR	^O	切换正交模式开或关 (Ctrl+O)
\	暂停以等候用户输入（不能用在加速键部分）	^P	切换 MENUCHO 开或关
-	转换 AutoCAD 命令及其后的关键字	^Q	回显所有提示、状态列表和打印输入 (Ctrl+Q)
+	延续菜单宏到下一行（如果是最后一个字符）	^T	打开或关闭数字化仪 (Ctrl+T)
=*	显示当前顶层的图像、下拉或快捷菜单	^V	切换当前视口 (Ctrl+V)
*^C^C	重复项的前缀	^Z	禁止在菜单项末尾自动增加 SPACEBAR 的空字符
\$	加载菜单部分或输入条件 DIESEL 宏表达式 (\$M=) 的特殊字符		

7.1 自定义菜单

- 特殊字符使用举例

- 例1：等候用户输入

- 在菜单项中有些命令（如画直线或圆）常需要从键盘或定标设备中输入数值。此时，在菜单宏表达式中需要加\来完成任务。如：

[Circle-1]circle \10

- **Circle-1**暂停请求用户输入圆心，然后从菜单中读入半径**10**，画出以指定点为圆心，以**10**为半径的圆。注意在\后没有空格，但在**circle**和\之间必须有一个空格。

7.1 自定义菜单

- 例2：取消上次执行的命令
- 在执行新的命令之前，要确保上次命令已经执行完或取消了。在**AutoCAD**命令操作环境下，按下两次**Ctrl+C**就可以取消命令。通常，一般的命令按一次**Ctrl+C**就可以取消；但也有些命令，如**text**命令，需要连续按两次**Ctrl+C**才能取消。为了保险起见，在菜单宏中使用`^C^C`取消前一次命令，其功能等同于在键盘上连续按两次**Ctrl+C**或**ESC**。如

`ID_Line [Line(&L)] ^C^C_line`

7.1 自定义菜单

- 1、菜单文件
- 2、加载菜单

AutoCAD中有5类菜单，哪5类？其中哪几类最重要？为什么？

这5类菜单文件是：

.mnu, .mns, .mnc, .mnl, .mnr

最重要的是菜单模板文件和源文件。

因为其它文件都可以由模板文件生成。而加载时先寻找源文件。

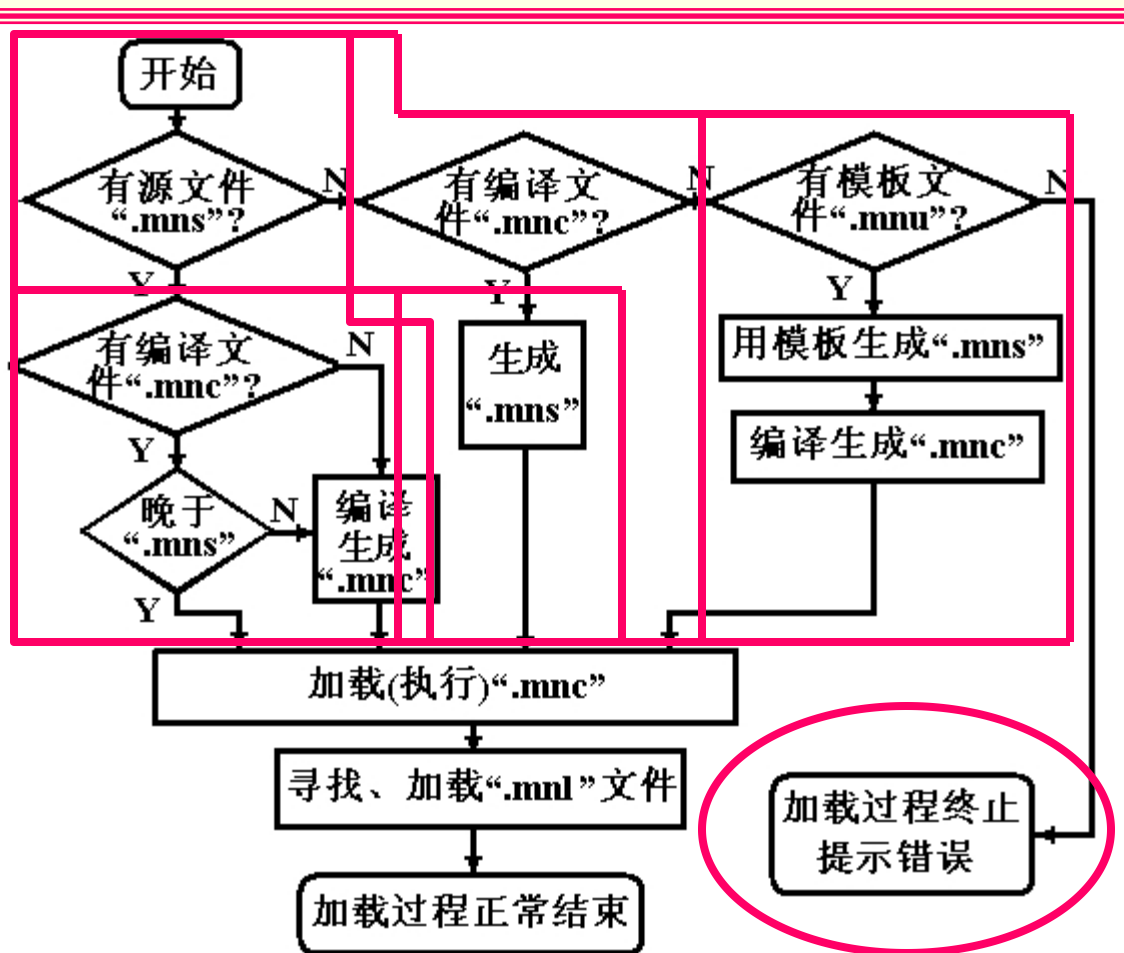
7.1 自定义菜单

1、菜单文件

2、加载菜单

第一步：找文件

- 1) 先找源文件。再找编译文件;或编译生成新的MNC文件;
- 2) 找MNC文件, 把它反编译出MNS文件;
- 3) 找mnu文件, 它编译出MNS和MNC文件;
- 4) 显示错误, 并提示用户输入别的菜单名。



AutoCAD 菜单加载过程

7.1 自定义菜单

1、菜单文件

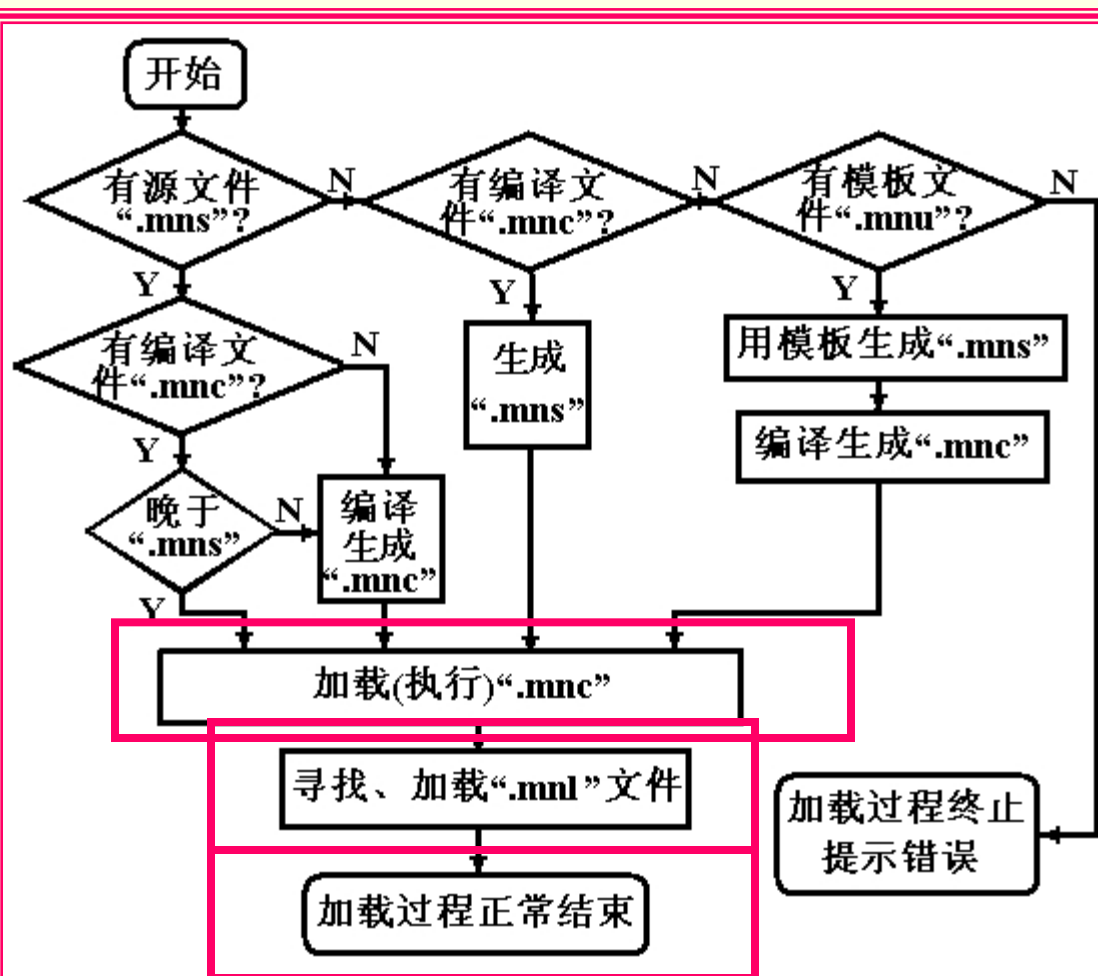
2、加载菜单

第一步：找mnc文件

第二步：加载（执行）MNC文件；

第三步：寻找、加载.MN1文件；

第四步：加载过程正常结束。



AutoCAD 菜单加载过程

7.1 自定义菜单

- 根据菜单加载的过程，如何使对于菜单的改动得以固定和保存？
- 如果对**菜单模板文件.mnu**进行了修改，并要让改动起作用，就必须在启动**AutoCAD**系统前将**.mns**和**.mnc**文件删除，或是重新加载**.mnu**模板菜单文件。
- 如果在**AutoCAD**中，**用AutoCAD的功能对菜单进行了修改**，这些变化自动记录在**.mns**文件中，为了使修改得以固定，在删除**.mns**文件之前，应将修改的内容移到**.mnu**文件的相应部位中，并将**.mnu**文件备份下来。如果这些改动是在基础菜单**acad.mnu**中，则应在**AutoCAD**重新安装后，用改动过的**acad.mnu**去覆盖**AutoCAD**原始的**acad.mnu**，并删除掉已存在的**acad.mnc**和**acad.mns**文件。
- 如果在**AutoCAD**启动后对菜单进行了修改，想要取消这些修改，恢复启动前的菜单状态，可以用**menu**命令，调入**acad.mnu**菜单模板文件。（**Menu**输入后会自动显示出所在路径。最好备份好**acad.mnu**）
- **C:\Users\lenovo\AppData\Roaming\Autodesk\AutoCAD 2010\R18.0\chs\Support\mymenu.mnu**

7.1 自定义菜单

菜单文件是文本文件，在记事本中输入保存即可。

1、菜单文件

2、加载菜单

3、应用举例

1) 设计菜单

```
// 注释行
// Mymenu 菜单文件 - 盘符: \acad2008 \support \
//                               Mymenu.mnu
***MENUGROUP=MyMenu
***POP1
**MyMenu
ID_MyMenu      [自定义(&Y)]
ID_Ruf          [粗糙度(&R)  Ctrl+R]^C^C_ccd1
                [--]
ID_Ln           [对数线(&L)  Ctrl+L]^C^C_Inx
ID_Sinx         [正弦线(&S)  Ctrl+S]^C^C_sinx
```

这个菜单中要执行的三个功能都是CAD中没有的，要自己设计。

AutoCAD 2008 - [Drawing1.dwg]

文件(F) 编辑(E) 视图(V) 插入(I) 格式(O) 工具(T) 绘图(D) 标注(N) 修改(M) 窗口(W) 帮助(H)

自定义(Y)

二维草图与注释

粗糙度(R) Ctrl+R

对数线(L) Ctrl+L

正弦线(S) Ctrl+S

7.1 自定义

1、菜单文件

2、加载菜单

3、应用举例

1) 设计菜单

2) 设计函数

设计函数要用
LISP编程。

```
;  
; Mymenu 菜单文件要加载的lisp文件 -  
; 盘符:\acad2000\support\sinx.lsp  
;
```

画正弦曲线的函数

```
(defun c:sinx( / a fai dfai x1 x p0 p1)  
  (setq a (getreal "\n Input the Amplitude: "))  
  (setq fai (getreal "\n Input the Phase: "))  
  (setq fai (* (/ fai 180.0) pi))  
  ;把相位角变成弧度  
  (setq p0 (list 0 (+ 100 (* a (sin fai)))))  
  ;计算起始点坐标  
  (command "line" "0,100" "400,100" "")  
  (command "line" "0,0" "0,200" "")  
  ;画坐标轴线  
  (setq x 1) ;设循环变量初值  
  (setq dfai (/ pi 20))  
  ;计算相位角增量  
  (while (<= x 40)  
    (setq fai (+ fai dfai))  
    (setq x1 (* x 10))  
    (setq y1 (+ (* a (sin fai)) 100))  
    (setq p1 (list x1 y1))  
    (command "line" p0 p1 "")  
    (setq p0 p1)  
    (setq x (+ x 1))  
  )  
) ; 画正弦曲线的函数结束
```

```

; 画对数曲线的函数 盘符:\acad2000\support\lnx.lsp
(defun c:lnx()
  (setq value (getreal "\n 请输入X值: "))
  (if(<= value 0)
    (progn
      (princ "\n错误: X值不能小于等于0! ")
      (princ)
    ) ;第一progn结束
    (progn
      (setq delta (/ value 20)) ;把绘图区间分为20份
      (setq x delta)
      (setq n 0)
      (setq p0 (list delta (log delta))) ;计算起始点坐标
      (while (<= n 19)
        (setq x (+ x delta) y (log x))
        (setq p1 (list x y))
        (command "line" p0 p1 "")
        (setq p0 p1)
        (setq n (+ n 1))
      ) ;while循环结束
      (command "line" (list 0 (min 0 (log delta)))
                (list 0 (max 0 (log value)))) ""
      (command "line" "0,0" (list value 0) "")
      (command "zoom" "w"
                (list 0 (min (log value) (log delta)))
                (list value (max (log value) (log delta))))
    )
  ) ;第二progn结束
) ;if(= value 0)结束
) ; 画对数曲线的函数结束

```

1、菜

2、加

3、应

1) 设

2) 设

7.1

1、菜单文件

2、加载菜单

3、应用举例

1) 设计菜单

2) 设计函数

; 标注表面粗糙度的函数 盘符:\acad2000\support\ccd.lsp

```
(defun c:ccd1()
```

```
(setq p0 (getpoint "请输入插入点的坐标:"))
```

```
(setq alf (getangle "请输入旋转角度:"))
```

```
(setq text0 (getstring "请输入粗糙度的Ra值:"))
```

;下面计算各绘图参数

```
(setq h0 (getvar "dimtxt")) ;读取系统变量dimtxt
```

```
(setq df (getvar "dimscale")) ; 读取系统变量dimscale
```

```
(setq a0 (* 1.6 h0 df))
```

```
(setq a1 (* 1.4 h0 df))
```

```
(setq rad (/ pi 180)) ;pi是AutoLISP内部定义的常数, 3.1415926
```

```
(setq alf1 (/ alf rad))
```

```
(setq bt1 (* (+ alf1 60) rad))
```

```
(setq bt2 (* (+ alf1 120) rad))
```

```
(setq bt4 (* (+ alf1 90) rad))
```

```
(setq x1 (+ (car p0) (* a0 (cos bt1))))
```

```
(setq y1 (+ (cadr p0) (* a0 (sin bt1))))
```

```
(setq p1 (list x1 y1))
```

```
(setq x2 (+ (car p0) (* a0 (cos bt2))))
```

```
(setq y2 (+ (cadr p0) (* a0 (sin bt2))))
```

```
(setq p2 (list x2 y2))
```

```
(setq x3 (+ (car p0) (* 2.0 a0 (cos bt1))))
```

```
(setq y3 (+ (cadr p0) (* 2.0 a0 (sin bt1))))
```

```
(setq p3 (list x3 y3))
```

```
(setq x4 (+ (car p0) (* 1.5 a1 (cos bt4))))
```

```
(setq y4 (+ (cadr p0) (* 1.5 a1 (sin bt4))))
```

```
(setq p4 (list x4 y4))
```

;下面开始绘图

```
(command "Line" p1 p2 p0 p3 "")
```

```
(if (or (and (<= alf -90) (>= alf1 - 240))
```

```
(and (<= alf1 280) (>= alf1 120))
```

```
) ;end or
```

```
(setq alf1 (+ alf1 180))
```

```
) ;endif
```

```
(command "text" "m" p4 h0 alf1 text0) ;以中点方式把p4作为插入点输出文本
```

```
) ;defun结束
```

7.1 自定义菜单

1、菜单文件

2、加载菜单

3、应用举例

1) 设计菜单

2) 设计函数

3) 加载程序

```
//  
//      Mymenu 菜单文件 - 盘符:  
//      \aacad2008\support\Mymenu.mnl  
(load "ccd")  
(load "sinx")  
(load "lnx")  
(princ "\n加载自定义菜单... Loaded.")  
(princ)  
//
```

为了把菜单中的功能与功能函数连接起来，必须要设计一个与菜单文件同名的，扩展名为：“**mnl**”的LISP函数加载文件。

7.1 自定义菜单

1、菜单文件

2、加载菜单

3、应用举例

1) 设计菜单

2) 设计函数

3) 加载程序

4) 加载菜单

- **AutoCAD**加载菜单有两个命令：**menu**和**menuload**。
- 基础菜单是**AutoCAD**使用菜单的基础，在**AutoCAD**中同时只能使用一个菜单文件作为基础菜单，**acad.mnu**是安装时缺省的基础菜单。
- **menu**命令专用于更换或重新加载一个基础菜单，**AutoCAD**启动时缺省的基础菜单是上一次通过**menu**命令调入的菜单。

7.1 自定义菜单

1、菜单文件

2、加载菜单

3、应用举例

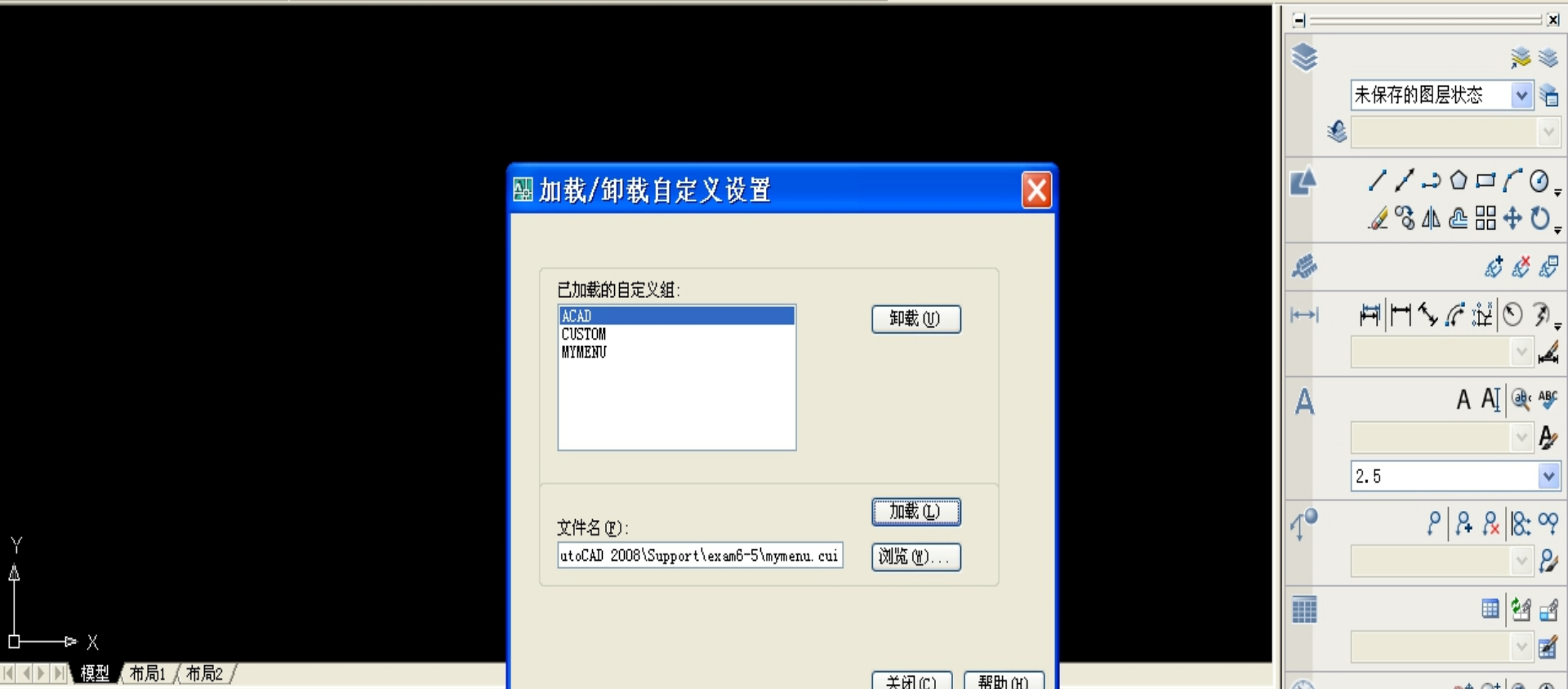
1) 设计菜单

2) 设计函数

3) 加载程序

4) 加载菜单

- 局部菜单和基础菜单在文件体系上没有区别。在基础菜单之上,可以在下拉菜单中附加上额外的菜单项,这种可附加的菜单就可称为**局部菜单**。
- 局部菜单使用**menuload**命令加载。
- **menuload**加载的局部菜单可以用**menuunload**命令将其卸载。
- 而用**menu**命令加载的基础菜单只能由下一个**menu**命令加载来替换。



加载自定义文件成功。自定义组: CUSTOM
加载自定义文件成功。自定义组: MYMENU
命令:
加载自定义菜单... Loaded.
命令: menuunload
已成功地卸载自定义文件。自定义组: MYMENU
命令: menuload
加载自定义文件成功。自定义组: MYMENU

7.1 自定义菜单

- 1、菜单文件
- 2、加载菜单
- 3、应用举例
- 4、步骤总结

1) 设计菜单	设计菜单文件 “.mnu”
2) 设计功能函数	设计功能函数 “.lsp”
3) 加载程序设计	设计加载程序 “.mnl”
4) 首次使用	加载菜单

这几个步骤中最难的是功能函数设计，它需要根据具体应用的需要来进行。

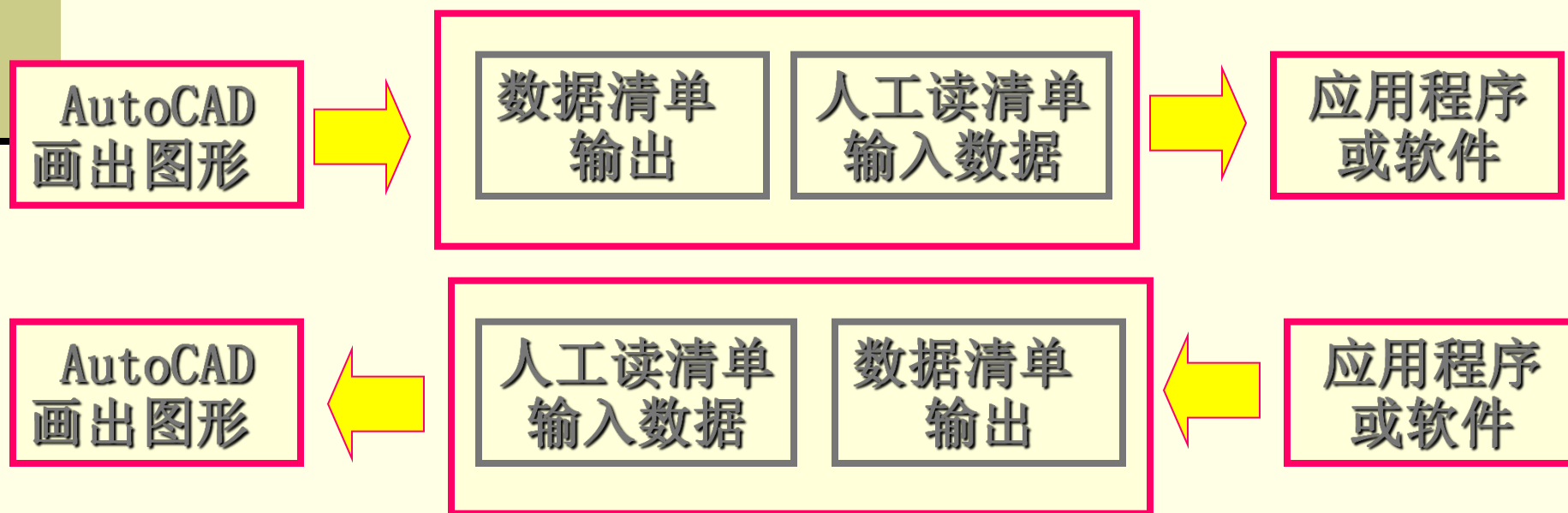
7.2 DXF文件

- **AutoCAD**是一种基础绘图软件，在实际应用中还必须对其进行二次开发，以适应实际应用的需要。
- 在计算机辅助设计中，有大量的设计计算工作，采用**FORTRAN**、**C**等高级语言比较能发挥这类语言的优势。但根据设计结果需要绘制工程图时，则**AutoCAD**绘图软件便能显出绘图的优势。
- **AutoCAD**虽是一个功能很强的图形软件包。但它的图形数据库是以**压缩的数据格式**存储的。所以，用户所编写的高级语言程序很难直接与之进行信息交流。为了能与用户用高级语言所编写的应用程序相连接，**AutoCAD**提供了与高级语言接口的功能。便于用户进行二次开发，以适应各种不同的应用环境。

7.2 DXF文件

1、数据交换

人工读取和输入数据是非常麻烦的，并且容易出错。
AutoCAD为了实现与其他高级语言程序之间图形信息的交换，定义了一个高级语言能够调用的“图形转换”文件的格式，这就是DXF文件。



7.2 DXF文件

1、数据交换

DXF(Drawing eXchang File)文件： 也称**图形数据交换文件**，供外部程序和图形系统或者不同的图形系统之间交换图形信息的文件。



7.2 DXF文件

1、数据交换

2、文件结构

DXF文件是一种带有“**DXF**”文件类型和专用格式文体的**ASCII**码的文本文件，它具有固定的格式，可以容纳**AutoCAD**所有的绘图信息，是**AutoCAD**与高级语言程序联接的一种重要形式。

DXF文件的最小组成单元是组(**Group**)，即**DXF**文件是由一系列的组所组成，每个组占两行。

一个组结构中：
第一行是组码；
第二行是组值。

```
0
SECTION
2
HEADER
9
$ACADVER
1
AC1014
9
$ACADMAINTVER
70
0
9
$INSBASE
10
0.0
20
0.0
30
0.0
. . .
0
SECTION
2
ENTITIES
0
LINE
5
24
100
AcDbEntity
8
0
. . .
0
ENDSEC
```

7.2 DXF文件

1、数据交换

2、文件结构

- **组码：**标记数据类型的整数。

既可表示组值类型，又可表示组的一般用途。组码的特定功能取决于它描述的是变量、表项还是实体。

- **组值：**数据的具体值，可能是整数，实数、或字符串。
其数据类型取决于组码的数值，即：

0~9为字符串，**10~59**为实型数，**60~79**为整型数。

因此，应用程序可根据组码得知组值的数据类型。

组值的坐标以十进制数表示(特别大时用科学表示法)，
角度以十进制度数表示，以+X轴方向为基准。

7.2 DXF文件

1、数据交换

2、文件结构

下面给出每一组码的所有功能，详见
P39.

- **0** 表示一个事物开始，如一个段、一个表、一个块、一条直线、一个圆等；
- **1** 一个文本，如字符串的值；
- **2** 名字，如段、表、块的名字；
- **3** 字符型数据的值，如线型说明部分、属性内容等；
- **5** 实体句柄；
- **6** 线型名；
- **7** 字样名；
- **8** 图层名；
- **9** 标题变量名；
- **10** 主X坐标(直线起点，字符串起点，圆心等)；
- **11~18** 其他X坐标；

7.2 DXF文件

1、数据交换

2、文件结构

- **20 Y坐标**(20以上的组码与10以上的组码是对应的，在文件中相继出现)；
- **21~28 其他Y坐标**；
- **30 主Z坐标**；
- **31~37 其他Z坐标**；
- **38 基面高度**；
- **39 厚度**；
- **40~48 高度、宽度、距离、比例因子等**；
- **49 重复性的值**；
- **50~58 角度**；
- **62 颜色号**；
- **66 实体跟随标记**；
- **70~78 整数**，如重复次数、各种标志或状态设置。
- 更多组码说明参见Visul LISP帮助“**DXF参考**”。

7.2 DXF文件

1、数据交换

2、文件结构

- DXF文件由7段和文件结尾组成。
- 段书写的基本单位是组，每段均有若干组构成，每组在文件中占两行，首行为组代码，第二行为组值。
- 有几个特殊的组作为文件分界符，用于表示段及表文件的开始与结束。
- 如：段头(SECTION)，段尾(ENDSEC)；
表头(TABLE)，表尾(ENDTAB)；
文件结束(EOF)。

7.2 DXF文件

1、数据交换

2、文件结构

DXF文件由7段和文件结尾组成。

7个段分别是：

- 1) 标题段 (Header)
 - 2) 类段 (Classes)
 - 3) 表段 (Tables)
 - 4) 块段 (Blocks)
 - 5) 实体段 (Entities)
 - 6) 对象段 (Objects)
 - 7) 预览段 (Thumbnailimage)
- 文件结尾 0 EOF

段的排列顺序不能改变。

每个段的结构为：

- 1) 段标记一组：
0 SECTION
- 2) 段名一组：
2 XXXXXXXX
- 3) 内容多组：
由许多组组成
- 4) 段结束一组：
0 ENDSEC

```
0
SECTION
2
ENTITIES
0
LINE
5
24
100
AcDbEntity
8
0
. . .
0
ENDSEC
```

7.2 DXF文件

- 1、数据交换
- 2、文件结构
- 3、各段信息

1) 标题段：标题段记录系统内存变量值和环境设置。由55个变量给出有关图形的各种参数和当前的设置，表明图形的总的信息，每个参数都有一个变量名和一个相应值。

这些内存变量和环境设置包括：版本号、改版号、图形插入点、绘图界限的最小点、最大点、捕捉间距、格栅间距等。

9组是变量名，1组是字符串，10组是X坐标……

```
0
SECTION
2
HEADER
9
$ACADVER
1
AC1014
9
$ACADMAINTVER
70
0
9
$INSBASE
10
0.0
20
0.0
30
0.0
9
$EXTMIN
10
99.431486
20
88.966729
30
0.0
. . .
0
ENDSEC
```

7.2 DXF文件

- 1、数据交换
- 2、文件结构
- 3、各段信息

2) 表段 (TABLES) :

本段含设置线型、图层、颜色、字体、视图大小等项定义, 内容如下:

视图 (VPORT) 表,
线型 (LTYPE) 表,
图层 (LAYER) 表,
字样 (STYLE) 表,
用户坐标系 (UCS) 表,
尺寸式样 (DIMSTYLE) 表,
应用程序标识 (APPID) 表。

```
0
SECTION
2
TABLES
0
TABLE
2
VPORT
5
8
100
AcDbSymbolTable
70
3
0
VPORT
5
29
100
AcDbSymbolTableRecord
100
AcDbViewportTableRecord
2
*ACTIVE
```

表: 表头 (6组)
+ 表内容 + 表尾
(1组)

总体: 段头 (2组)
+ 表 + 段尾 (1组)

```
70
0
10
0.0
20
0.0
0
... ..
0
ENDTAB
0
TABLE
2
XXXX
.....
0
ENDTAB
.....
其它表
0
ENDSEC
```

7.2 DXF文件

总体：段头（2组） + 多块 + 段尾（5组）

1、数据交换

2、文件结构

3、各段信息

3) 块段包含图形中所有块的记录。包括用户定义的命名块和HATCH命令及尺寸标注命令所生成的无名块。记录内容包括块的总体信息（名称、类型、基点）和组成块的成员实体信息。

块：块头 + 实体记录 + 块尾（1组）

```
0  
SECTION  
2  
BLOCKS
```

块名

块类型

插入基点

实体记录1

实体记录2

块尾

```
0  
BLOCK  
5  
xxx  
100  
AcDbEntity  
8  
0  
100  
AcDbBlockBegin
```

```
2  
ZHA12  
70  
0  
10  
0.0  
20  
0.0  
30  
0.0  
3  
ZHA12  
1  
xxxx  
0  
LINE
```

```
.....  
0  
CIRCLE  
.....  
0  
ENDBLK
```

```
0  
BLOCK  
5  
xxxx  
.....  
0  
ENDBLK
```

其它块记录

```
5  
xxxx  
100  
AcDbEntity  
8  
0  
100  
AcDbBlockEnd  
0  
ENDSEC
```

总体：段头（2组）+ 实体记录 + 段尾（1组）

7.2 DXF文件

- 1、数据交换
- 2、文件结构
- 3、各段信息

4) 实体段对所有图形实体及各种插入块进行详细说明，该段定义了每个实体的种类、所在图层名、颜色、线型、厚度、实体描述字以及有关的几何数据（包括任何引用的块）。

注意：

实体段最简单；
实体段最有用；

```
0  
SECTION  
2  
ENTITIES  
0  
LINE  
5  
24  
100  
AcDbEntity  
8  
0  
100  
AcDbLine  
10  
-0.647762  
20  
0.0  
30  
0.0  
.. ..  
  
0  
CIRCLE  
5  
25  
.. ..  
  
.. ..  
  
0  
ENDSEC
```

实体1记录

实体2记录

其它实体

7.2 DXF文件

1、数据交换

2、文件结构

3、各段信息

4、实体记录

1) 直线:

组码: 组值含义

5: 句柄, 实体数字标识;

100: 子类标记;

8: 图层名;

10: 起点X;

20: 起点Y;

30: 起点Z

11: 终点X;

21: 终点Y;

31: 终点Z

```
0
LINE
5
24
100
AcDbEntity
8
0
100
AcDbLine
10
-0.647762
20
0.0
30
0.0
11
134.734523
21
0.0
31
0.0
```


7.2 DXF文件

1、数据交换

2、文件结构

3、各段信息

4、实体记录

1) 直线:

2) 圆:

组码: 组值含义

5: 句柄, 实体数字标识;

100: 子类标记;

8: 图层名 (字符串);

10: 圆心点X;

20: 圆心点Y;

30: 圆心点Z

40: 半径 (实数);

```
0  
CIRCLE  
5  
25  
100  
AcDbEntity  
8  
0  
100  
AcDbCircle  
10  
246.149609  
20  
3.235294  
30  
0.0  
40  
54.356801
```


7.2 DXF文件

1、数据交换

2、文件结构

3、各段信息

4、实体记录

1) 直线:

2) 圆:

3) 弧:

组码: 组值含义

5: 句柄, 实体数字标识;

100: 子类标记;

8: 图层名 (字符串);

10: 圆心点X;

20: 圆心点Y;

30: 圆心点Z;

40: 半径 (实数);

50: 起始角;

51: 终止角;

```
0
ARC
5
2B
100
AcDbEntity
8
0
100
AcDbCircle
10
192.55524
20
183.08831
30
0.0
40
56.915746
100
AcDbArc
50
313.020033
51
139.912146
```

7.2 DXF文件

1、数据交换

2、文件结构

3、各段信息

4、实体记录

1) 直线:

2) 圆:

3) 弧:

还有许多实体，如矩形、多义线、文本等等，有关它们的组码及含义请在DXF的帮助文件中查找。

7.3 生成DXF文件

1、DXF文件的生成命令

- 用户在**AutoCAD**的图形编辑状态下使用**dxfout**命令可生成一个当前图形的图形交换文件。同样，进入**AutoCAD**，使用**dxfin**命令可以将图形交换文件转换成**DWG**文件，直接在屏上显示所要绘制的图形。

1. **dxfout**命令——写DXF文件

命令格式：

Command: dxfout

File name: (文件名)；输出的图形转换文件名

2. **dxfin**命令——读DXF文件

命令格式：

Command: dxfin

File name: (文件名)；输入要读的图形转换文件名

7.3 生成DXF文件

2、构造DXF文件

- 如果已经有了图形，要生成图形交换文件，可利用上述的**dxfout**命令。
- 但如果图形并不存在，比如是在设计过程中构思的图形，或者是利用其他程序的计算结果，需要把设计结果输入到**AutoCAD**中去画图，期望能生成图形。
- 这就需要把你的图形数据构造成**DXF**文件，然后用**AutoCAD**去打开这个文件，把图形显示出来或画到图纸上。
- 这种情况下就提出了需要**生成DXF文件**的问题。

7.3 生成DXF文件

2、构造DXF文件

我们可以利用文本编辑程序，按照图形交换文件所要求的格式来手工生成，但这不是自动化的，不符合计算机辅助设计的要求。因此，需要掌握如何用高级语言程序(如**FORTRAN**、**C语言**)来自动生成图形交换文件。这样就可以使绘图模块和计算模块连接起来，以实现**AutoCAD**与高级语言程序接口。

DXF文件是相当复杂的，一个完整的**DXF**文件一般都有几千行。但在一般情况下，这几千行记录中只有少数几十行是最有用的，并且**AutoCAD**允许把用不到的记录省略。——这就是基本原则。

7.3 生成DXF文件

2、构造DXF文件

构造DXF文件的基本原则：

- 1) 用不到的段都可以省略；
- 2) 段头、段尾不可省；
- 3) 文件尾不可省；

最少有一段组成；
一般都有实体段。

7.3 生成DXF文件

利用文本编辑程序，按照**DXF**要求的格式手工生成：

0	0	0	0	0
SECTION	LINE	CIRCLE	ARC	ENDSEC
2	8	8	8	0
ENTITIES	A	B	C	EOF
	10	10	10	
	0.000000	3.000000	0.000000	
	20	20	20	
	0.000000	3.000000	3.000000	
	11	40	40	
	5.000000	3.000000	3.000000	
	21		50	
	5.000000		270.000000	
			51	
			0.000000	

7.3 生成DXF文件

注意：头尾不可少，关键字不可有前空格，组码要正确。

2、构造DXF文件

```
#include <stdio.h>
void write_line(fp, xs, ys, xe, ye)
FILE *fp;
float xs, ys, xe, ye;
{
    fprintf(fp, "0\nLINE\n 8\nA\n");
    fprintf(fp, "10\n%f\n", xs);
    fprintf(fp, "20\n%f\n", ys);
    fprintf(fp, "11\n%f\n", xe);
    fprintf(fp, "21\n%f\n", ye);
}

void write_circle(fp, xc, yc, r)
FILE *fp;
float xc, yc, r;
{
    fprintf(fp, "0\nCIRCLE\n 8\nB\n");
    fprintf(fp, "10\n%f\n", xc);
    fprintf(fp, "20\n%f\n", yc);
    fprintf(fp, "40\n%f\n", r);
}
```

```
void write_arc(fp, xc, yc, r, s, e)
FILE *fp;
float xc, yc, r, s, e;
{
    fprintf(fp, "0\nARC\n 8\nC\n");
    fprintf(fp, "10\n%f\n", xc);
    fprintf(fp, "20\n%f\n", yc);
    fprintf(fp, "40\n%f\n", r);
    fprintf(fp, "50\n%f\n", s);
    fprintf(fp, "51\n%f\n", e);
}

void main()
{
    FILE *fp;
    char fname[12]="mydxf.dxf";
    fp=fopen(fname, "w");
    fprintf(fp, "0\nSECTION\n");
    fprintf(fp, "2\nENTITIES\n");

    write_line(fp, 0.0, 0.0, 5.0, 5.0);
    write_circle(fp, 3.0, 3.0, 3.0);
    write_arc(fp, 0.0, 3.0, 3.0, 270.0, 0.0);

    fprintf(fp, " 0\nENDSEC\n 0\nEOF\n");
    fclose(fp);
}
```


7.3 生成DXF文件

3、读取DXF文件

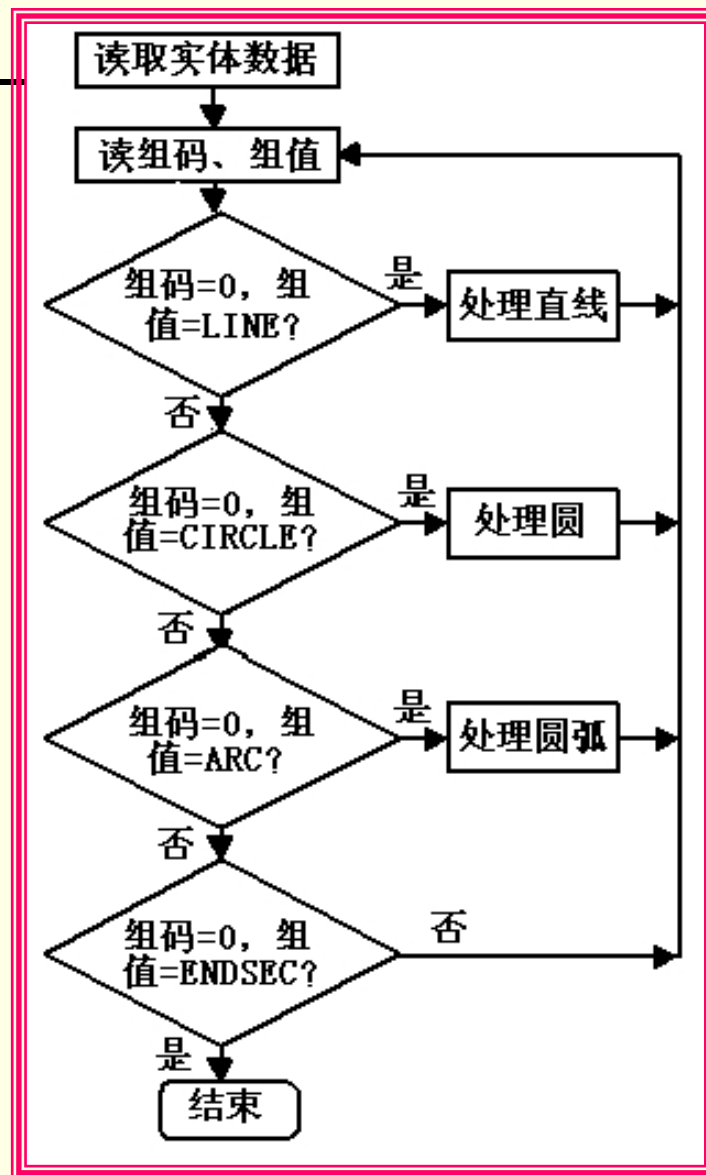
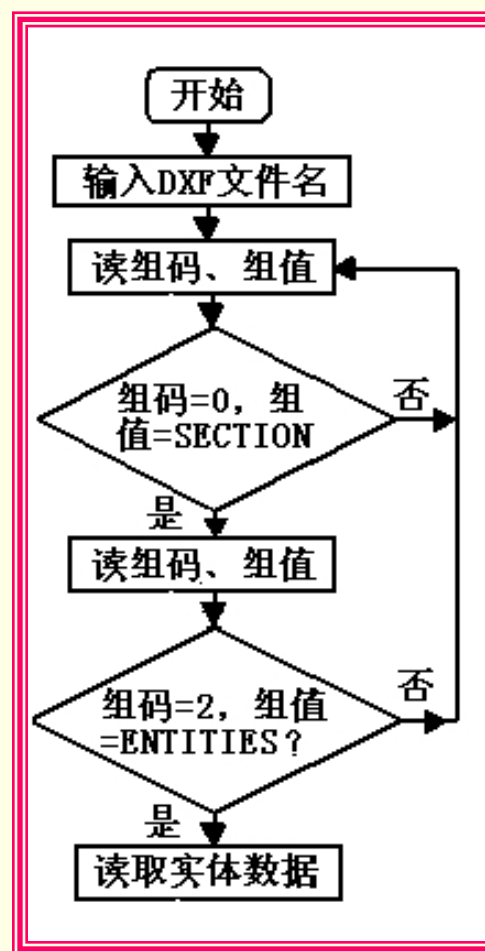
- 假如我们在屏幕上设计好了某个图形，要利用高级语言程序进行下一步的计算，或者当我们对利用某种前处理的模块输出的图形不满意，而对它做了某些修改时，同样需要进行下一步的计算。
- 二者都要求用程序去读取图形交换文件中的信息，特别是读取那些对计算来说是十分必要的图形实体的信息。这就是**AutoCAD**与高级语言程序接口的另一种形式。
- 为了简化讨论，下面只就如何编写程序去读取文件中的简单的图形实体来说明其方法，实际中的应用可在此基础上扩充。

7.3 生成DXF文件

3、读取DXF文件

读取DXF文件比构造DXF文件要复杂一些，对于不含图块及特殊类对象的基本文件，图形数据提取工作可以分两步进行：

- 1) 寻找实体段；
- 2) 实体数据处理。



7.3 生成DXF文件

3、读取DXF文件

处理基本图形实体的具体程序要根据图形实体的类型来具体设计。对于直线关键是要寻找起点X坐标（组码=10）和终点坐标（组码=11）。

具体程序框图如下。

圆和圆弧实体信息的提取，完全可以参照直线进行。

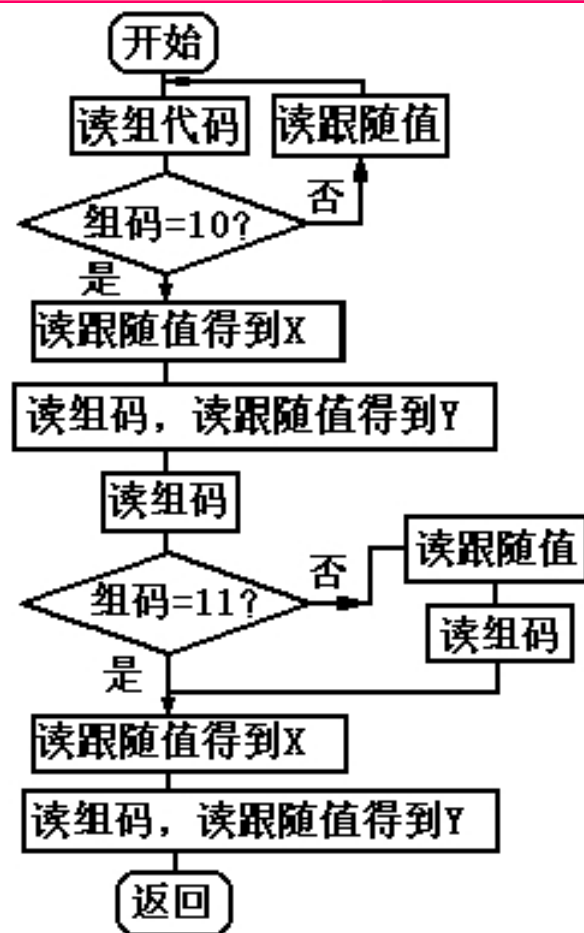


图6-8-2 处理直线的程序流程